```
it "displays if logged in" do
  login_as users(:quentin)
  expect(logged_in?).to be_truthy
  expect(if_logged_in { "logged in" }).to eq("logged in")
end
```

The first test asserts that the block is not invoked, so the helper returns nil. The second asserts that the block is invoked, just returning the value passed into the block.

You have to be a little careful here because these tests are just testing the helper method's return value, not what is sent to the output stream. The output-stream part is a side effect of the process, but it is stored in a variable called output_buffer, which you can access via testing. So you could write the preceding tests as follows:

```
it "does not display if not logged_in" do
  expect(logged_in?).to be_falsy
  if_logged_in { "logged in" }
  expect(output_buffer).to be_nil
end
```

```
it "displays if logged in" do
  login_as users(:quentin)
  expect(logged_in?).to be_truthy
  if_logged_in { "logged in" }
  expect(output_buffer).to eq("logged in")
end
```

If for some reason your helper method requires a specific instance variable to be set, cut that out immediately; it's a bad idea. However, if you must use an instance variable in your helper and you want to test it in RSpec, use the assigns method, as in expect(assigns(:project)).to eq(Project.find(1)). That tests that there is an instance variable named @project that has been set to a specific project.

Testing Controllers and Requests

The biggest change in Rails 5 testing when compared to earlier versions is the deprecation of some controller-testing functionality. In this section we'll first look at the RSpec features in Rails 5, then we'll go back and look at how controller testing worked in previous versions of Rails.

For most of this book I've focused on just what Rails 5 has to offer, and not spent time comparing Rails 5 to previous versions. I'm going to make a partial exception here and discuss the deprecated controller features as part of this section, for two reasons: